

# 向量搜索找得到碎片走不到背后的实体

## Introducing Document Intelligence: From documents to a knowledge graph, right inside Aura

**本篇范围声明：**这是 Neo4j 官方博客的一篇**产品发布公告**（标注阅读时长 **11 分钟**，比前两天那个三部曲长得多，也实在得多）。它宣布 Document Intelligence 在 Neo4j Aura 里进入预览。下面的「全文翻译」覆盖正文全部七节；导航栏、页脚与「开始使用」的链接段不译。它是**发布公告不是论文**：有架构说明和明确的能力边界，但没有评测数据、没有和竞品的对比、没有失败案例。

### 核心内容

**Q：这个产品要解决什么？** A：作者开篇给的观察是：过去几个季度里 agent 式 workflow 从试验走向生产，**客户反复带来同一个需求**：

**他们的 agent 需要推理的数据里，90% 躺在文档里**（10-K 财报、合同、研究论文、政策手册、内部维基），而他们的 AI 系统能读完这些文档的每一个字，却从来看不到把这些文档串起来的关系。

紧接着是全文最锋利的一句：

**Vector search finds similar fragments. It cannot traverse the entities behind them.** 向量搜索找得到相似的碎片。它走不到碎片背后的实体。

**Q：为什么以前做不成？** A：作者说「知识图谱能补上这个缺口，**难的一直是怎么走到那一步**」，然后列了拦路的五样：**模式设计、本体决策、抽取提示词、胶水代码、几周的迭代**。

**Document Intelligence 的卖点就是把这五样都去掉：**在 Aura 控制台里走一个引导式流程，**不需要图建模背景，不用写抽取代码**——把 PDF 丢给 agent，用大白话描述你想抽什么，看着图自己长出来。

**Q：架构是什么？三个部件。**

| 部件                            | 做什么                                                                        | 产出                                                   |
|-------------------------------|----------------------------------------------------------------------------|------------------------------------------------------|
| 文档处理器<br>(document processor) | 解析文件， <b>按版面感知的方式切块</b> ，把块和它们的 embedding 一起存下来                            | 词法图 (lexical graph)：文档连到块、块连到块， <b>每个块都可按向量相似度检索</b> |
| 模式生成器 (schema generator)      | <b>跨文档采样</b> ，把你的自然语言提示当作指引，提出实体类型、关系类型和属性                                 | 一份 <b>落在图画布上的模式提案</b> ，可以直接编辑，也可以和 agent 对话来改        |
| 实体抽取器 (entity extractor)      | 拿着模式和你的指令对 <b>每一个块</b> 跑 LLM 抽取； <b>经一个合并步骤去重</b> ；把每个抽出的实体 <b>链回它被提到的</b> | 实体层                                                  |

部件

做什么

产出

那些块

去重那一步作者给了具体例子：让你不会得到 `Apple Inc.`、`Apple, Inc`、`APPLE` 三个分开的节点。

抽取作为后台导入任务运行，可以关掉页面回头再看。作者写道：「对小文档集这只是方便；对大的，这是让这套流程能用起来的前提。」

Q：最终得到的是什么形状？ A：一张混合图（hybrid graph），两层住在一起：

- 词法层：用于段落级检索和溯源
- 实体层：用于回答 agent 真正需要回答的结构化问题

两层出自同一套流程。

Q：那个「一个提示词」到底写什么？ A：作者说整个体验里最重要的一个输入框就是它。模式已经告诉了 agent 要抽哪些类型，提示词是你给它上下文的地方：关注点、你想让这张图回答的业务问题、以及你要它按名字认出来的具体实体。

三个真实例子：

`Focus on products, services, segments, and competition.`

`Pay particular attention to year-over-year changes in segment revenue.`

`Focus on financial risk factors.`

`Recognize these portfolio companies by name: Stripe, Cohere, Anthropic, Hugging Face.`

`Focus on the methods used, the datasets evaluated, and the limitations the authors call out.`

`Recognize Stanford, MIT, DeepMind, and Anthropic as institutions.`

`Ignore acknowledgments and funding statements.`

作者对这一段的评价：「那一小段话，就是一张通用的图和一张能回答你业务真正在问的问题的图之间的差别。上下文工程，浓缩进一个输入框。」

Q：能力边界在哪？这一节最实在。 A：预览版的硬限制：

- 每个任务最多 20 篇文档，每篇 10 到 20 页
- 格式：PDF、DOCX、TXT、EPUB、HTML、Markdown，且必须是以文字为主的
- 多栏版面、表格、图片密集的文档「在路上」（还不支持）

## 背景补充 / 点评

「向量搜索找得到碎片，走不到碎片背后的实体」这句话值得单独记住

这是全文最有分量的一句，而且它准确。

向量检索的机制是：把查询和文本块都映射到同一个向量空间，取距离最近的几个块。它的返回单位是块，块之间没有边。所以当问题是「把这几份文件里共有的风险因素找出来」时，向量检索能找到几个都提到风险的块，但它没有任何机制去发现「这两个块提到的是同一个实体」。

这正是知识图谱那一层要补的东西：实体抽取 + 去重 + 链回块，把散在不同文档里的同一个实体归并成一个节点，块之间才有了可走的路。

三个部件的分工，实际上是在拆一个大家都会踩的坑

很多人做「文档转知识图谱」时，把「切块」「定模式」「抽实体」揉成了一步：直接拿一个提示词让 LLM 从原文里吐三元组。

这样做的问题是模式不稳定：第一篇文档抽出 OFFERS，第二篇抽出 PROVIDES，第三篇抽出 SELLS，三个是同一个意思但成了三条边。

Document Intelligence 的做法是先定模式再抽——模式生成器跨文档采样先提出一套类型，人确认后，抽取器拿着这套固定的模式去每个块上抽。这一步把「模式漂移」这个问题在架构上排除掉了。

「文档到块到实体」这个三层结构，CSRS 是同一个形状

Neo4j 说的词法图 (documents → chunks → entities, chunks 上带 embedding)，和 CSRS 的入库链路是同一个形状。CSRS 也是解析 → 切块 → 存 embedding → 抽实体进 Neo4j。

差别在于谁定模式：

- Document Intelligence：模式生成器从文档里采样提出来，人在画布上改，每个任务一套
- CSRS 的 cognify：Node.type 是自由文本，抽的是通用无 schema 三元组，没有「先定模式」这一步

这个差别的后果就是上面说的模式漂移。而昨天读的那篇 GRA 论文提出的方案丙（入库时按维度读成一张表），本质上也是在解决同一件事——都是「先定结构再抽」，只是 Neo4j 让 LLM 提结构，方案丙让人按行业定维度表。

「多栏版面、表格、图片密集的文档在路上」——这条限制要重点看

这是 Neo4j 自己承认的边界，而且它和 CSRS 遇到的问题是同一类。

CSRS 那边的具体形态是：docx 的图片能抽出来（我实测过，5 张一张不丢），但 docling 不给 page\_no（Word 的分页要渲染才确定），所以 prov 是空的，图绑不到 chunk。

Neo4j 的说法更笼统（「on the way」），但指向同一个根因：版面信息在解析层就丢了，后面所有需要位置的功能都接不上。

这说明这不是某一家实现得不好，是当前文档解析这一层的普遍状态。判断任何「文档转知识图谱」方案时，版面能力应该单独问一遍，不能默认包含在「支持 PDF/DOCX」里。

路线图里有一条值得单独拎出来：Context graph

作者列的六条路线图里，最后一条是：

**Context graph.** Schema provenance, lineage, and the decisions that built your knowledge graph are captured as a separate graph alongside it, so the result is reproducible and the AI's reasoning is auditable. 上下文图。模式的来源、血缘，以及构建这张知识图谱时做过的那些决策，被捕获成一张独立的图放在它旁边，让结果可复现、让 AI 的推理可审计。

这一条说的是「把建模决策本身也存成图」，而这正是我现在手工在做的事——那四份本体文档（选型、理论、RDF 笔记、样例）里记的全是「为什么这样定」「哪条被撤回了」「谁在哪天拍的板」。

Neo4j 把它放在路线图的最后一条，说明这件事没那么容易；但它被列出来本身说明，「决策的记录」被认为是知识图谱的一部分，不是文档工作。

血脉那一节透露的产品演进路径

作者专门写了一节「A heritage worth acknowledging」（一段值得承认的血脉），三个阶段：

| 阶段     | 形态                                                 | 代价                |
|--------|----------------------------------------------------|-------------------|
| 2022 起 | LangChain 流水线，代码为主                                 | 完全的控制权，代价是可观的工程投入 |
| 中间     | Neo4j Labs 的 LLM Graph Builder，一个 UI 把同一套模式做给更广的受众 |                   |
| 现在     | Document Intelligence，Aura 里的一等能力                  | 同样的成熟模式，带生产支持     |

作者明确说「最成熟的一批客户仍然用第一种方式建，而当深度定制重要时那就是对的选择」——这句话不常见，产品公告里承认自己的新产品不适合最高端的客户。

我认为要打折扣的两处

第一，「无需图建模背景」这个卖点和「模式质量」之间有张力。

模式生成器能提出一套类型，但它提得对不对，需要有人判断。作者自己给的 Apple 10-K 例子里，模式生成器提出了 Company、Product Line、Product Category、Segment、Service、Operating System、Platform 七个类型和 OFFERS、HAS\_SEGMENT、IS\_A\_TYPE\_OF 等七条关系——这套模式合不合理，恰恰需要图建模背景才判断得了。

比如 IS\_A\_TYPE\_OF 这条边，如果它连的是「产品类别是一种产品线」，那是子类关系应该用继承；如果连的是「iPhone 是一种产品」，那是实例关系。这两者在图里长得一样，但含义完全不同——这正是我这几天在基石 B8、判据 J2 里反复处理的那个区分。

「不需要建模背景」只是说不需要写代码，不是说不需要建模判断。

第二，没有任何质量数据。

全文没有一个数字回答：抽出来的实体准确率多少、去重的误合并率多少、模式生成的提案有多少需要人改。对一个「让 LLM 从文档里抽结构」的产品，这三个数是最该给的，一个都没有。

作为预览版这可以理解，但读的时候要清楚：这篇给的是「能做什么」，不是「做得多好」。

## 对我的启示

第一条，「先定模式再抽」这条原则可以直接用来评估 CSRS 的 cognify。

CSRS 现在抽的是通用无 schema 三元组，`Node.type` 是自由文本。按这篇的三部件划分，CSRS 缺的是「模式生成器」那一环——没有跨文档采样提出一套固定类型的步骤，每篇文档各抽各的。

这直接对应我在合规检查评估里给的方案丙（入库时按维度读成一张表，文档没写到的维度也留一行记「未涉及」）。当时我写「CSRS 现在没有这条链，cognify 抽的是通用无 schema 三元组，抽不出带维度带属性带值的清单，要新建」——Neo4j 用产品化的方式验证了这条链是必要的，而且他们把它做成了三个独立部件而不是一个提示词。

第二条，「版面能力要单独问一遍」是一条可复用的评估动作。

Neo4j 明说多栏、表格、图片密集的文档还不支持；CSRS 那边 docx 的页码拿不到、图绑不到 chunk。两家的表述不同，根因相同：版面信息在解析层丢了。

固化成一个提问：评估任何文档处理方案时，「支持 PDF/DOCX」这句话之后要追问三件事——多栏版面认不认？表格结构保不保留？图片有没有位置信息？这三个是分开的能力，不能打包默认。

第三条，那个「一个提示词」的设计，值得抄进合规检查方案。

Document Intelligence 把「关注什么、要回答什么业务问题、按名字认哪些实体」压进一个自然语言输入框，而不是拆成一堆配置项。作者管这叫「上下文工程，浓缩进一个输入框」。

这个形状对合规检查的「维度表」可以借用：与其让客户填一张结构化的维度配置，不如先给一个输入框让他用大白话说「这份说明书里我关心适应症、用法用量、禁忌，特别注意剂量数字」，再由系统提出维度表让他改。

理由和 Neo4j 一样：结构化配置要求填的人先懂结构；自然语言 + 提案 + 人改，只要求他懂业务。

第四条，路线图里的「Context graph」提醒我一件事：建模决策的记录目前是纯手工的。

我这几天做的所有裁决——属性和关系的分界改了三次、`rdfs:domain` 被撤回、B8 的表述被修正、J6 四问的确立——全部只记在 markdown 文档里，`ontology/` 那几个 YAML 里一条都没有。

这意味着：加载器能校验「引用的名字存在」（AXP3），但校验不了「这个建模选择的理由是什么、什么时候定的、谁定的」。`umu:kind` 是我这两天加的第一个把建模选择显式登记进图的机制，但它只记了结果（是属性还是关系），没记理由。

Neo4j 把这件事列进路线图，说明它是个真问题，不是我一个人的洁癖。

## 文中金句

"90% of the data their agents need to reason over lives in documents ... and their AI systems can read every word of those individual documents without ever seeing the relationships that tie them together." 他们的 agent 需要推理的数据里 90% 躺在文档里……而他们的 AI 系统能读完这些文档的每一个字，却从来看不到把这些文档串起来的关系。

"Vector search finds similar fragments. It cannot traverse the entities behind them." **向量搜索找得到相似的碎片。它走不到碎片背后的实体。**

"The hard part has always been getting there: schema design, ontology decisions, extraction prompts, glue code, weeks of iteration." 难的一直是**怎么走到那一步**：模式设计、本体决策、抽取提示词、胶水代码、几周的迭代。

"That short paragraph is the difference between a generic graph and one that answers the questions your business actually asks. Context engineering, condensed into a single field." **那一小段话，就是一张通用的图和一张能回答你业务真正在问的问题的图之间的差别。上下文工程，浓缩进一个输入框。**

"The entities and relationships were always there, buried in flat text where no traversal could reach them." **那些实体和关系一直都在，只是埋在扁平的文本里，没有任何遍历能够到它们。**

"Many of our most sophisticated customers still build that way, and that is the right choice when deep customization matters." **我们最成熟的一批客户仍然用那种方式（写代码）来建，而当深度定制重要时，那就是对的选择。**

## 全文翻译

### 开篇

Over the past few quarters, we have seen agentic workflows move from experimentation to production across the enterprise.

过去几个季度里，我们看到 agent 式工作流在企业中**从试验走向生产**。

客户越来越多地带来同一个需求：**他们的 agent 需要推理的数据里，90% 躺在文档里**（10-K 财报、合同、研究论文、政策手册、内部维基），而他们的 AI 系统能读完这些文档的每一个字，却从来看不到把这些文档串起来的关系。**向量搜索找得到相似的碎片。它走不到碎片背后的实体。**

A knowledge graph closes that gap. The hard part has always been getting there: schema design, ontology decisions, extraction prompts, glue code, weeks of iteration.

**知识图谱补上这个缺口。难的一直是怎么走到那一步**：模式设计、本体决策、抽取提示词、胶水代码、几周的迭代。**这正是我们要解决的。**

今天我们宣布 **Document Intelligence**，现已在 Neo4j Aura 的所有云服务商和所有档位上进入预览。它通过 Aura 控制台里的一个引导式流程把文档变成可查询的知识图谱。**不需要图建模背景，不用写抽取代码。把 PDF 丢给**

agent，用大白话描述你想抽什么，看着图自己长出来。做完之后，你可以直接在这个 agent 里和你的图对话、提问。

## 为什么需要 Document Intelligence

Every team building with AI hits the same wall. Language models are powerful reasoners, but they need clean, structured, connected data to reason well. Without it, their context window fills with noise, and accuracy collapses.

**每个用 AI 做东西的团队都会撞上同一堵墙。** 语言模型是强大的推理者，但它们需要干净的、结构化的、连接起来的数据才能推理得好。没有这些，它们的上下文窗口被噪声填满，准确率就崩了。

它们伸手去够的那些文档，**从来就不是为图形状的检索而组织的**。当一个 agent 需要回答「把这家公司经营的每一条产品线 and 它卖进的每一个细分市场都给我看」或者「这几份申报文件里有哪些共有的风险因素」时，**向量搜索和关键词索引不够用**。

The agent needs a graph: something it can use to retrieve the relevant context surgically, explain where answers came from, and stop burning tokens on noise.

**agent 需要一张图：**一个能让它外科手术式地取出相关上下文、解释答案从哪来、并且不再把 token 烧在噪声上的东西。

作者列了过去一年从数据和 AI 团队那里听到的问题：**怎么把一个装满 PDF 的文件夹变成知识图谱，而不用另起一条 ML 流水线？怎么告诉抽取我真正在乎的东西（产品、服务、风险），而不用额外写提示词？怎么在做这些的同时保持我的数据保护、安全与合规姿态不变？怎么在任何东西落进数据库之前先预览抽出了什么？怎么在我已经跑图工作负载的那个环境里完成这一切？怎么让我的 agent 同时查询我的结构化企业数据和我的文档，并遍历连接它们的那些关系？**

## 认识 Document Intelligence

### 第一步，从你的文档生成一个图模型。

在 Aura 控制台里打开 Document Intelligence，点「创建图模型」，加一个数据源。可以丢本地文件，也可以通过「云数据源」标签页连一个存储桶。**发布时支持 PDF、DOCX、TXT、EPUB、HTML 和 Markdown。**

点「生成模型」，Document Intelligence 会**解析你的文档、跨文档采样、并提出一个扎根于你内容实际所含之物的起始图模型**。你也可以跳过自动生成，**直接对内置的 agent 描述你想要的模型**——对很多领域来说，那是更快的路。

**对一份 Apple 的 10-K**，两页文字返回的是 Company、Product Line、Product Category、Segment、Service、Operating System、Platform 这些类型，以及 OFFERS、HAS\_SEGMENT、OPERATES、IS\_A\_TYPE\_OF、BASED\_ON\_OS、COMPETES、PROVIDES\_SERVICE 这些关系。模型落在一张**可缩放、可检视、可直接编辑**的图画布上。

### 第二步，用一个提示词引导抽取。

The most important field in the experience is one text box.



**整个体验里最重要的一个输入框就是它。** 模型已经告诉了 agent 要抽哪些类型，**提示词是你给它上下文的地方**：关注点、你想让这张图回答的业务问题、以及你要它按名字认出来的具体实体。（三个例子见「核心内容」一节。）

That short paragraph is the difference between a generic graph and one that answers the questions your business actually asks. Context engineering, condensed into a single field. No prompt tuning, no eval harness, no extraction pipeline to maintain.

**那一小段话，就是一张通用的图和一张能回答你业务真正在问的问题的图之间的差别。上下文工程，浓缩进一个输入框。不用调提示词，不用评测框架，没有抽取流水线要维护。**

**第三步，预览抽出的图，然后导入 Aura。**

切到画布底部的「图预览」，你会看到模型被从你文档样本中抽出的真实实体填满。**对那份 10-K，中心是 Apple Inc.，周围是 iPhone、iPad、Apple TV 4K、HomePod、AirPods、Apple Music、Apple Card、Apple Pay、Apple Fitness+、App Store 和云服务；细分市场有美洲、大中华区、日本、欧洲和亚太其他地区；还有竞争对手和平台。全都连着，全都从两页文字里抽出来。**

The entities and relationships were always there, buried in flat text where no traversal could reach them.

**那些实体和关系一直都在，只是埋在扁平的文本里，没有任何遍历能够到它们。**

Document Intelligence 把它们浮出来，**并且通过它们共享的实体把你的文档连起来**。两页 10-K 变成一张可查询的图。一百份合同变成一张图，让你能问出任何单份合同都回答不了的问题。

满意之后点「运行导入」。**模型、抽出的实体、关系，以及底层的词法层（文档到块到实体，块上带 embedding）全部写进你的 Aura 实例。**

## 它怎么工作

Document Intelligence 坐在你的文档和你的知识图谱之间，**三个部件协同工作。**

**文档处理器**解析你的文件，以版面感知的方式切块，把块和它们的 embedding 一起存成一张**词法图**：文档连到块、块连到块，**每个块都可按向量相似度检索。**

**模式生成器**跨你的文档采样，把你的自然语言提示当作指引，提出实体类型、关系类型和属性，**扎根于你的内容**。提案落在图画布上，在任何抽取跑起来之前，你可以直接编辑它，或者和 agent 对话来打磨它。

**实体抽取器**拿着你的指令**对每一个块跑模式引导的 LLM 抽取，经一个合并步骤去重**（这样你不会得到 Apple Inc.、Apple, Inc 和 APPLE 三个分开的节点），**并把每个抽出的实体链回它被提到的那些块。**

Extraction runs as a background import job, so you can close the tab and come back to a finished graph. For small document sets, that is convenient. For larger ones, it is what makes the workflow usable at all.

**抽取作为后台导入任务运行**，所以你可以关掉标签页，回来时图已经建好。**对小文档集这只是方便；对大的，这是让这套流程能用起来的前提。**

The result is a hybrid graph in your Aura instance: a lexical layer for passage-level retrieval and provenance, and an entity layer for the structured questions your agents actually need to answer. Both



live together, and both come out of the same workflow.

结果是你 Aura 实例里的一张混合图：一个用于段落级检索和溯源的词法层，一个用于回答你的 agent 真正需要回答的结构化问题的实体层。两者住在一起，两者出自同一套流程。

### 一段值得承认的血脉

Neo4j 从 2022 年就在做「从文档到知识图谱」。最早的模式是代码为主的：LangChain 流水线，给团队完全的控制权，代价是可观的工程投入。

Many of our most sophisticated customers still build that way, and that is the right choice when deep customization matters.

我们最成熟的一批客户仍然用那种方式来建，而当深度定制重要时，那就是对的选择。

Neo4j Labs 随后把同一套做法打包成 LLM Graph Builder，一个让更广受众也能用上这套模式的 UI。

Document Intelligence 是下一步：同样经过验证的模式，作为一等的、受支持的能力带进 Aura，与你其余的图工作流集成。对多数刚起步的团队，这是我们现在推荐的路；对那些已经用不下 Labs 版本、需要生产支持的客户，这是一条清晰的跑道。

### 接下来：从聚焦的预览到平台能力

Document Intelligence 今天作为一个聚焦的预览版发布。这是刻意的。

今天的限制：每个任务最多 20 篇文档，每篇 10 到 20 页，且是以文字为主的格式（PDF、DOCX、TXT、EPUB、HTML、Markdown）。多栏版面、表格和图片密集的文档在路上。规模也是。

路线图六条：

| 条目     | 内容                                                             |
|--------|----------------------------------------------------------------|
| 规模     | 每任务先到数百、再到数千篇文档，配并行处理与队列管理。预览客户最常提的诉求，也是接下来投入的重点               |
| 程序化访问  | API、CLI 和 MCP server 端点，让你交互式跑的同一套流程也能从流水线、IDE 或 agent 里跑      |
| 复杂文档   | 多栏版面、表格、带文字的嵌入图片。为那些看起来不像文本、更像报告的文档做版面感知解析                     |
| 半结构化数据 | 从带文本列的 CSV 开始，那是结构化与非结构化世界之间最常见的桥                              |
| 可组合架构  | 自带模型、自带文档处理器、自带实体解析。默认会持续变好，接缝对需要定制的团队开放                       |
| 上下文图   | 模式的来源、血缘，以及构建这张知识图谱时做过的那些决策，被捕获成一张独立的图放在它旁边，让结果可复现、让 AI 的推理可审计 |

### agent 化企业的知识层

Agents are only as good as the context they can reason over, and that context comes from three places: the operational graphs you already maintain, the structured data in your warehouse, and the documents that hold your domain knowledge.

**agent 的上限就是它能推理的上下文的上限**，而那个上下文来自三个地方：**你已经在维护的运营图、你数仓里的结构化数据、以及承载你领域知识的那些文档**。Neo4j Aura 是横跨这三者的平台。

- **AuraDB** 是你最有价值的图数据之家，提供你运营工作负载需要的高密度关系与毫秒级延迟。
- **Virtual Graph** 把你数仓里已有的结构化数据开放给图推理，**不用复制、不用迁移**。
- **Document Intelligence** 解锁躺在文档里的那 90% 企业数据，**几分钟内把一个装满 PDF 的文件夹变成可查询的知识图谱**。

**把这些能力放在一个平台上一共用。最有图成熟度的企业将会这样建。**

## 参考链接

- [Introducing Document Intelligence: From documents to a knowledge graph, right inside Aura](#) — 本文原文 (Neo4j 博客)
- [Neo4j Aura 控制台](#) — Document Intelligence 在左侧导航里
- [LLM Graph Builder](#) — Neo4j Labs 的前代产品，本文血脉一节提到的中间阶段
- [What is GraphRAG](#) — 导入后可接的下游用法
- [Model Context Protocol](#) — 路线图「程序化访问」里提到的 MCP server 端点